

Re-running Architecture & Design analysis on FSDKC via GitHub OAuth (cloud-only). Discovering tools and fetching current `main` structure. Fetching repo structure and key source trees from `main` for a fresh KPI pass. Walking application source trees next — code search returned empty. Fetching controller and key source files for KPI measurement and evidence excerpts. I have enough evidence. Writing the full report, then publishing it on a new feature branch and opening a PR. Writing the complete architecture report, then publishing it via GitHub. Pushing the report to the feature branch via GitHub MCP. Uploading via GitHub Contents REST API (`gh api`), then opening the PR with MCP. Re-run complete against `main` @ `481814f` . Report: `docs/discovery/01-architecture-design.md` on branch `discovery/arch-design-20260724T162813` .
PR: <https://github.com/shende-shweta/FSDKC/pull/17>

Delta

Path	Action
<code>docs/discovery/01-architecture-design.md</code>	Created/updated on feature branch
<code>agent-runs/20260724T162813_tursg8/01-architecture-design.md</code>	Pipeline artifact
<code>agent-runs/20260724T162813_tursg8/delta-report.md</code>	Delta log

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Repository Pattern (H3), Direct SQL/ORM in Controllers (H6), Domain Boundary Violations (H8), Shared Database Coupling (H9), and Dual-Stack Duplicated Domain Logic (H10).

EXECUTIVE SUMMARY

Klearcom is a multi-layer monolith (Laravel 12 API, parallel Express `dev-api`, React 19 SPA) with Discovery and Connect modules named in folders but not enforced as bounded contexts — `backend/app/Modules/{Discovery,Connect}` and `frontend/src/modules/*` contain only `AGENTS.md` stubs. The dominant risk is **change amplification from missing repository/service boundaries**: controllers and Express handlers call Eloquent models and an in-memory `store` directly, while KPI/tree/reachability formulas are duplicated across PHP, Node, and the UI. Layers covered: **backend** (Laravel `backend/app` ~25 PHP files + `dev-api/src` 6 JS files) and **frontend** (`frontend/src` ~15 TS/TSX/JSX files). No mobile/CLI layers. Highest-severity hotspots are missing repositories (H3), cross-domain dashboard/legacy coupling (H8/H9), and dual-stack duplicated domain logic (H10). Frontend is comparatively healthier on LOC but still hard-codes API paths in pages and retains legacy class/error-throw patterns (F5).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Re
---	---------	-------------	------	----------	-----------	----------	----

H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	~83 LOC (6 Laravel API controllers)	
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	16 (4 Laravel + 12 Express handlers on <code>store</code>)	
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	~52 Eloquent/ <code>store</code> access sites; 0 repositories	
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (<code>LegacyDataMapper</code> , <code>store.js</code> <code>buildTree</code>)	
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	~31% of MariaDB access outside controllers	
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest ~280 LOC <code>server.js</code>)	
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8+ (Dashboard/Legacy/RealTime/dev-api cross Discovery+Connect)	
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	100% of domain tables (4/4) read by cross-cutting code	
H10	Dual-Stack Duplicated Domain Logic (additional)	Duplicate formula/algorithm copies across stacks (Good 0 · Moderate 1–3 · High Risk >3)	0	1–3	>3	8 copies across 3 formula clusters	
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	~95 LOC across 9 components/pages	
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 pages/components with hard-coded paths via shared <code>api</code> client	
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest <code>ConnectPage</code> ~222 LOC)	
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 levels; small Zustand <code>uiStore</code>	
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 (<code>LegacyMonitorPoller</code> class + <code>LegacyDashboardWidget</code> throw)	

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 Missing Service Layer	Add Discovery/Connect/Dashboard application services; stop Eloquent/ <code>store</code> use from controllers/handlers	MODERATE	HIGH
H3 Missing Repository Pattern	Introduce repository interfaces + Eloquent/in-memory implementations; route all persistence through them	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Replace <code>LegacyDataMapper</code> extract mapping; move <code>buildTree</code> out of <code>store.js</code> into Discovery domain	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Move Eloquent/Mongo queries out of HTTP layer into repositories/adapters	HIGH RISK	CRITICAL
H8 Domain Boundary Violations	Enforce real module packages; dashboard/legacy consume published interfaces only	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Declare table ownership; cross-context access via APIs/ACL, not shared Eloquent models	HIGH RISK	CRITICAL
H10 Dual-Stack Duplicated Domain Logic	Canonicalize reachability/tree/KPI formulas; delete PHP/Node duplicate copies	HIGH RISK	CRITICAL
F5 Legacy / Inconsistent Component Patterns	Migrate class poller to hooks; add ErrorBoundary; unify TSX conventions	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers and Express handlers become thin adapters; Discovery/Connect workflows are testable via application services without HTTP.
- Persistence is swappable (Eloquent ↔ in-memory ↔ alternate DB) behind repositories, enabling deterministic unit tests.
- Bounded contexts stop silent cross-domain breakage when Connect or Discovery schemas change.
- Single-source domain formulas eliminate Laravel vs `dev-api` drift on KPIs and reachability alerts.
- Frontend legacy patterns and missing boundaries stop uncaught UI failures and interval leaks as modules grow.